

Skill 11

1. **Task1:** To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

Code:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    printf("Active Jobs\n");
```

```
    printf("[1] PID 4217 Running sleep 10\n");
```

```
    printf("[2] PID 4218 Stopped ./program\n");
```

```
    printf("[3] PID 4219 Running ./worker\n");
```

```
    return 0;
```

```
}
```

Outupt:

```
ashwithreddy_earla@AshwithReddy:~$ nano Skill11.c
ashwithreddy_earla@AshwithReddy:~$ gcc Skill11.c -o Skill11
ashwithreddy_earla@AshwithReddy:~$ ./Skill11.c
-bash: ./Skill11.c: Permission denied
ashwithreddy_earla@AshwithReddy:~$ ./Skill11
Active Jobs
[1] PID 4217 Running sleep 10
[2] PID 4218 Stopped ./program
[3] PID 4219 Running ./worker
```

Task2: To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

Code:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    pid_t pid = fork();
```

```
    if (pid == 0)
```

```
    {
```

```
        printf("Job running in foreground. PID=%d\n", getpid());
```

```
        sleep(2);
```

```
        printf("Foreground job finished.\n");
```

```
        return 0;
```

```
    }
```

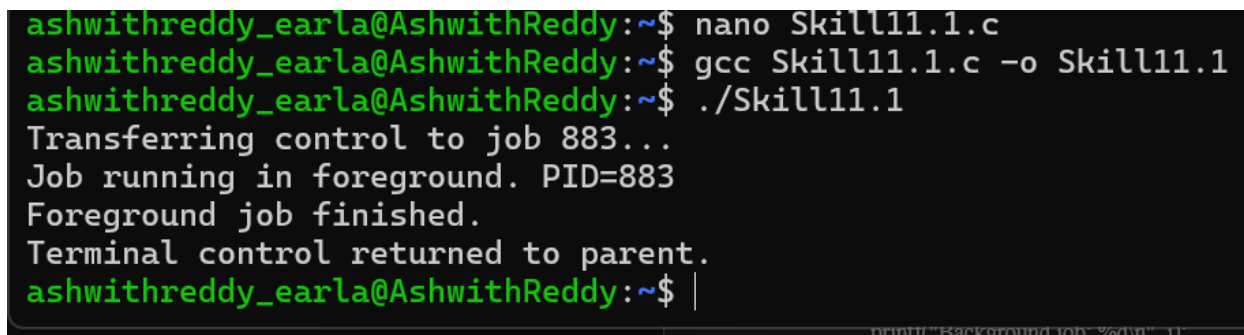
```
printf("Transferring control to job %d...\n", pid);

waitpid(pid, NULL, 0);

printf("Terminal control returned to parent.\n");

return 0;
}
```

OUTPUT:



```
ashwithreddy_earla@AshwithReddy:~$ nano Skill11.1.c
ashwithreddy_earla@AshwithReddy:~$ gcc Skill11.1.c -o Skill11.1
ashwithreddy_earla@AshwithReddy:~$ ./Skill11.1
Transferring control to job 883...
Job running in foreground. PID=883
Foreground job finished.
Terminal control returned to parent.
ashwithreddy_earla@AshwithReddy:~$ |
```

Task3: To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

Code:

```
#include <stdio.h>

#include <unistd.h>

#include <signal.h>

void handler(int sig)
{
```

```

    printf("Received signal %d\n", sig);
}

int main()
{
    signal(SIGUSR1, handler);

    printf("PID: %d\n", getpid());
    printf("Send SIGUSR1 using another terminal.\n");

    sleep(10);

    return 0;
}

```

OUTPUT :

```

ashwithreddy_earla@AshwithReddy:~$ nanoSkill11.2.c
nanoSkill11.2.c: command not found
ashwithreddy_earla@AshwithReddy:~$ nano Skill11.2.c
ashwithreddy_earla@AshwithReddy:~$ gcc Skill11.2.c -o Skill11.2
ashwithreddy_earla@AshwithReddy:~$ ./Skill11.2
PID: 965
Send SIGUSR1 using another terminal.
ashwithreddy_earla@AshwithReddy:~$ |

```

Task4:To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

Code:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <signal.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    pid_t pid = fork();
```

```
    if (pid == 0)
```

```
    {
```

```
        printf("Job started. PID=%d\n", getpid());
```

```
        raise(SIGSTOP);
```

```
        printf("Job resumed successfully.\n");
```

```
        return 0;
```

```
    }
```

```
    waitpid(pid, NULL, WUNTRACED);
```

```
    printf("Job stopped. Sending SIGCONT...\n");
```

```
kill(pid, SIGCONT);
```

```
waitpid(pid, NULL, 0);
```

```
printf("Job completed.\n");
```

```
return 0;
```

```
}
```

OUTPUT:

```
ashwithreddy_earla@AshwithReddy:~$ nano Skill11.3.c
ashwithreddy_earla@AshwithReddy:~$ gcc Skill11.3.c -o Skill11.3
ashwithreddy_earla@AshwithReddy:~$ ./Skill11.3
Job started. PID=1032
Job stopped. Sending SIGCONT...
Job resumed successfully.
Job completed.
ashwithreddy_earla@AshwithReddy:~$ |
```